

Prof. Dr. Matthias Wählisch
Faculty of Computer Science | Chair of Distributed and Networked Systems

Advanced Computer Networks

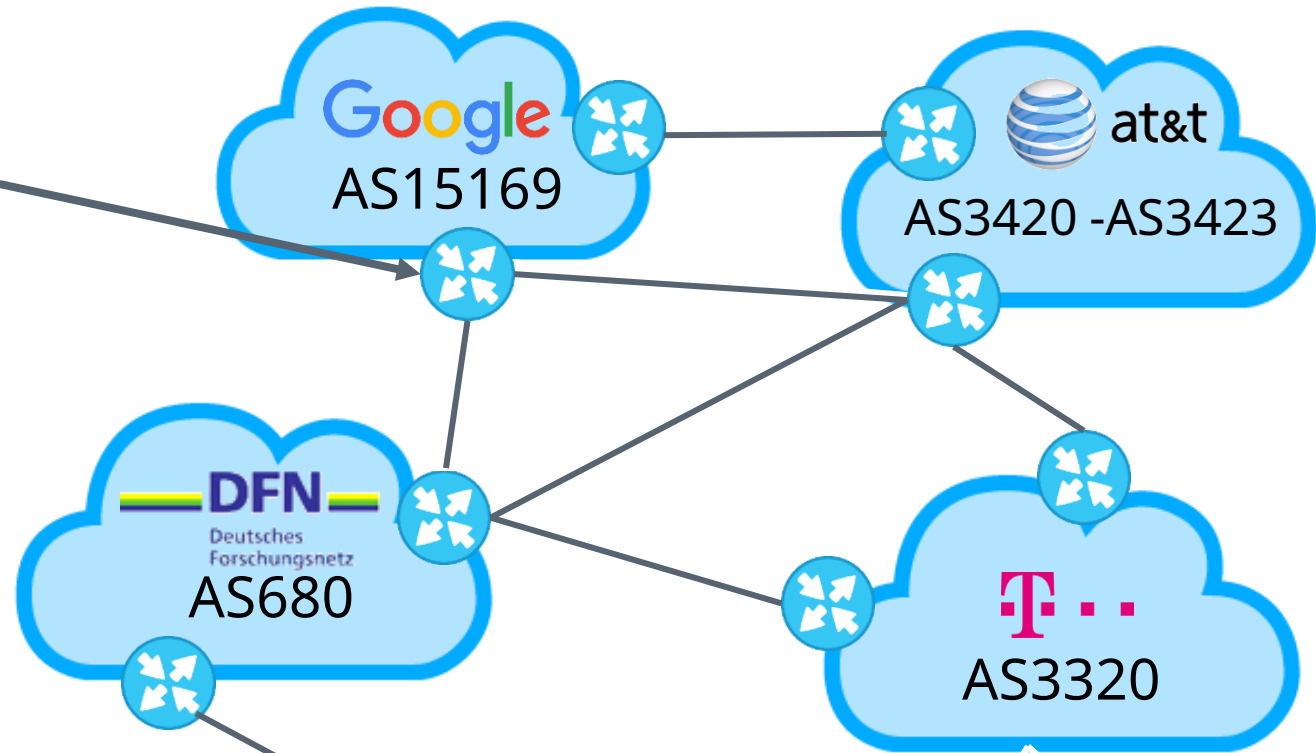
IPv6

What happened last lecture?

Inter-domain Routing Security

Autonomous systems = One domain

141.76.0.0/16 via DFN



IP prefixes

141.76.0.0/16

IP blocks

141.76.40.0/24

141.76.41.0/24

Hijacking of address space

When the core protocol for Internet routing (BGP) was designed security was not considered.



IP prefixes

141.76.0.0/16

IP blocks

141.76.40.0/24

141.76.41.0/24

Summary of potential hijacking and route leak options

Origin hijack

Sub-prefix
hijack

Path
manipulations

Policy
violations

Protection concepts proposed by the IETF

Route Origin Validation

ROV
(RFC 6811)

Mapping of IP prefixes to origin AS necessary
Including cryptographic proof
Prefix owner should be able to authenticate *Origin AS(es)*

Path Validation

BGPsec
(RFC 8205)

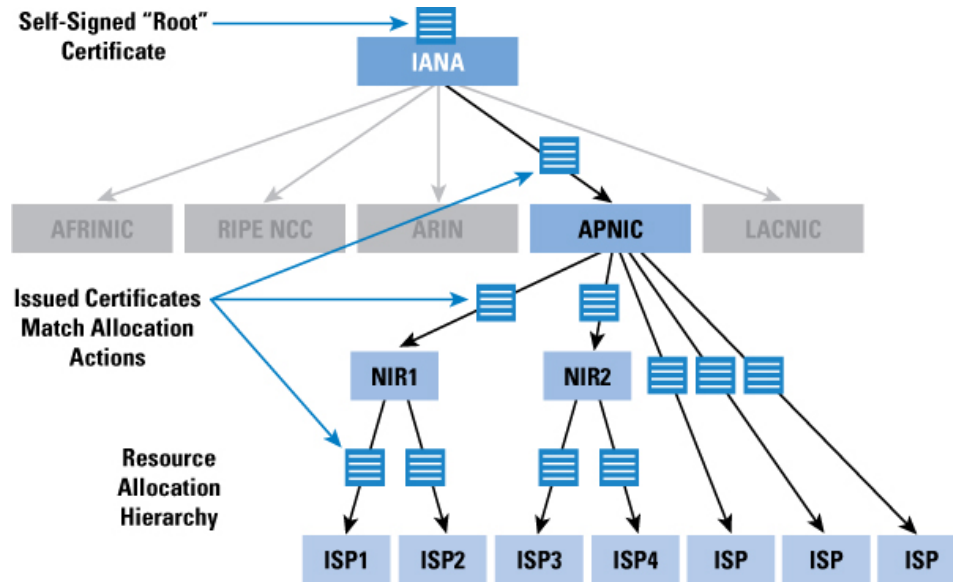
BGP path information are cryptographically secured
Paths will be signed

Policy Validation

ASPA
(draft-ietf-sidrops-aspa-verification)

Binds provider ASes to a customer AS

The Resource Public Key Infrastructure (RPKI) is the basic solution proposed in the IETF



Source: Cisco

RPKI is a system that allows to attest the usage of IP addresses and ASNs (i.e., Internet resources) and thus includes cryptographically provable certificates.

Certificate hierarchy reflects IP-/AS-allocation in the Internet

Currently, each RIR creates a self-signed root certificate

Implementation of the RPKI started January 2011.

All RIRs participate!

Currently, we lack a single root of trust. There are five trust anchors.

A ROA is not a certificate



Certificate

Proofs **ownership** of an
IP prefix or an AS number



ROA

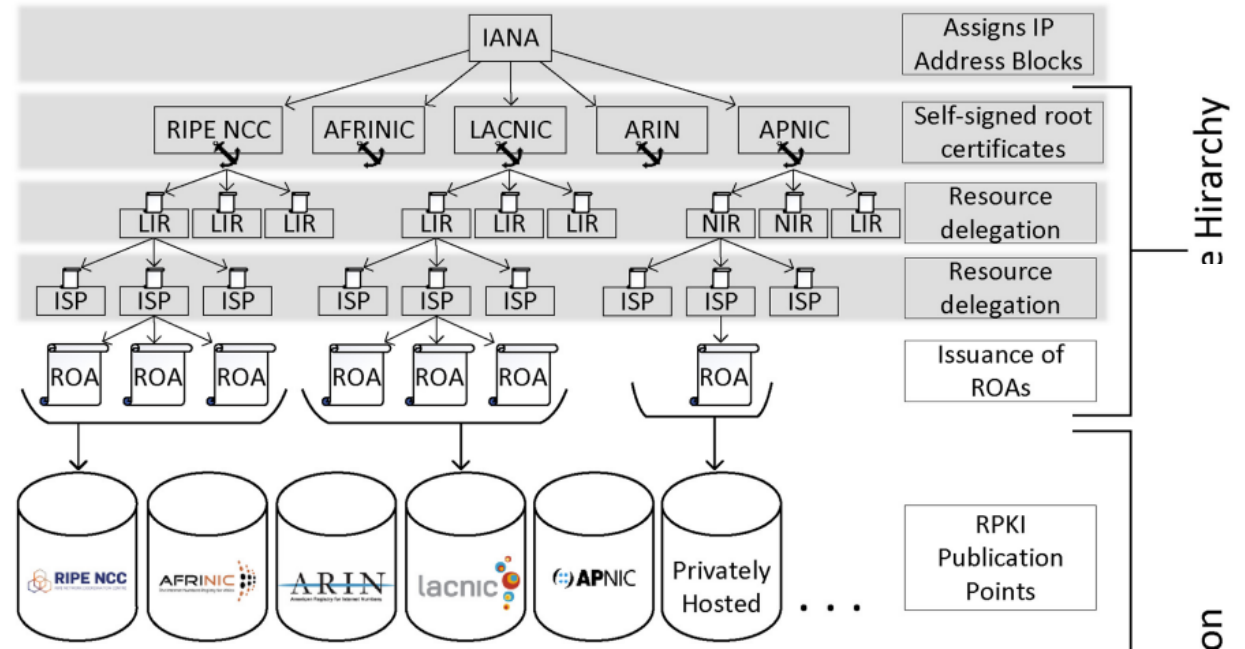
Proofs that an AS is **allowed
to announce** IP prefix(es)

RPKI architecture

There are two options to operate an RPKI repository

Hosted model, all certificates and ROAs are stored and managed by a RIR.

Delegated model, maintenance of the RPKI is delegated to third parties. Owners of Internet resources can run their own Certificate Authority (CA) to manage certificates and ROAs of IP prefixes. Running a publication point is also possible but not required.



RIPE and APNIC support both models.

Prefix origin validation using RPKI consists in two steps

(1) Validation of
ROAs



(2) Validation of
BGP Updates

Performed at external cache

Performed at BGP router
No additional cryptographic
operations necessary

How does the RPKI data comes to the BGP router?

ROAs and ROV. How do you validate a route in principle?

Valid ROAs

10.20.0.0/16-17 {AS10,
AS20}
10.20.30.0/24-24 {AS10}

BGP announcements

10.20.0.0/16: AS20 VALID

10.20.1.0/24: AS10 INVALID

10.20.0.0/16: AS100 INVALID

1.2.3.0/24: AS10 NOT FOUND

Check if AS is allowed to announce IP prefix = check the corresponding ROA

Consider all valid ROAs

Find those ROAs that cover the announced prefix based on classless-inter domain routing (CIDR)

Compare covering ROAs with route announcement

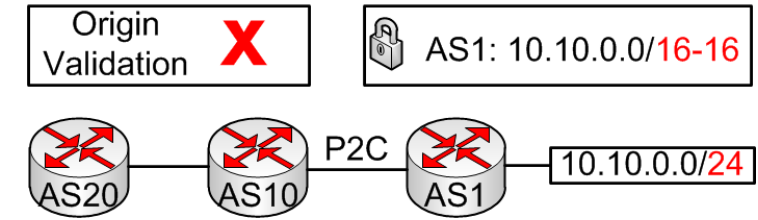
There are three states: VALID, INVALID, NOT_FOUND

RPKI implements a positive attestation

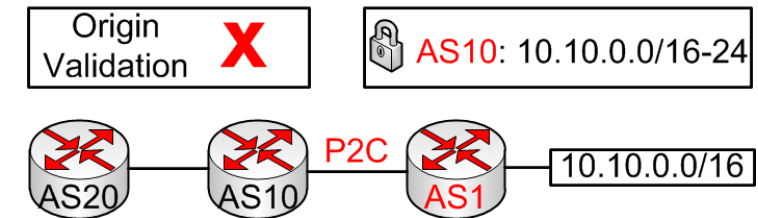
If a ROA for a prefix exists, announcements of all origin ASes that are not included or exceed the prefix length will be considered INVALID.

Classification of potential ROA misconfigurations

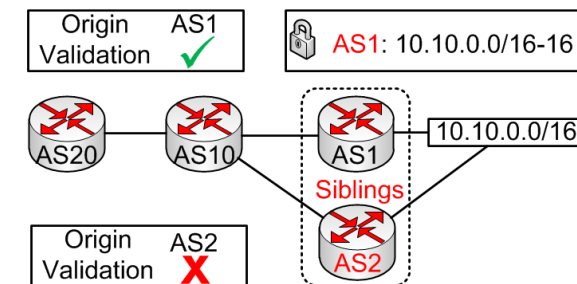
1. Valid origin, announced prefix is more specific



2. Provider does not consider customer



3. Additional ASes of a company are not authorized



Example configuration on Cisco router, deploying different preferences depending on the ROV outcome

Route map named rtmap-PEX1-3 sets a local preference of 50 for invalid prefix/AS pairs, 100 for not-found prefix/AS pairs, and 200 for valid prefix/AS pairs.

The local preference values are set for incoming routes from the neighbor at 10.0.102.1.

The neighbor at 10.0.102.1 is an eBGP peer.

```
router bgp 65000
  address-family ipv4 unicast
  neighbor 10.0.102.1 route-map rtmap-PEX1-3 in
  bgp bestpath prefix-validate allow-invalid
  !
  route-map rtmap-PEX1-3 permit 10
    match rpki invalid
    set local-preference 50
  !
  route-map rtmap-PEX1-3 permit 20
    match rpki not-found
    set local-preference 100
  !

  route-map rtmap-PEX1-3 permit 30
    match rpki valid
    set local-preference 200
  !

  route-map rtmap-PEX1-3 permit 40
```

Reasons for not deploying RPKI

Political reasons

- RIR are trust anchors
- Local law may instruct RIR to revoke certificates
- ROAs become invalid
- Out of control of the operator

Business reasons

- RPKI implements a positive attestation model
- ISPs have to add prefix-AS relation in advance
- Might conflict with business policies

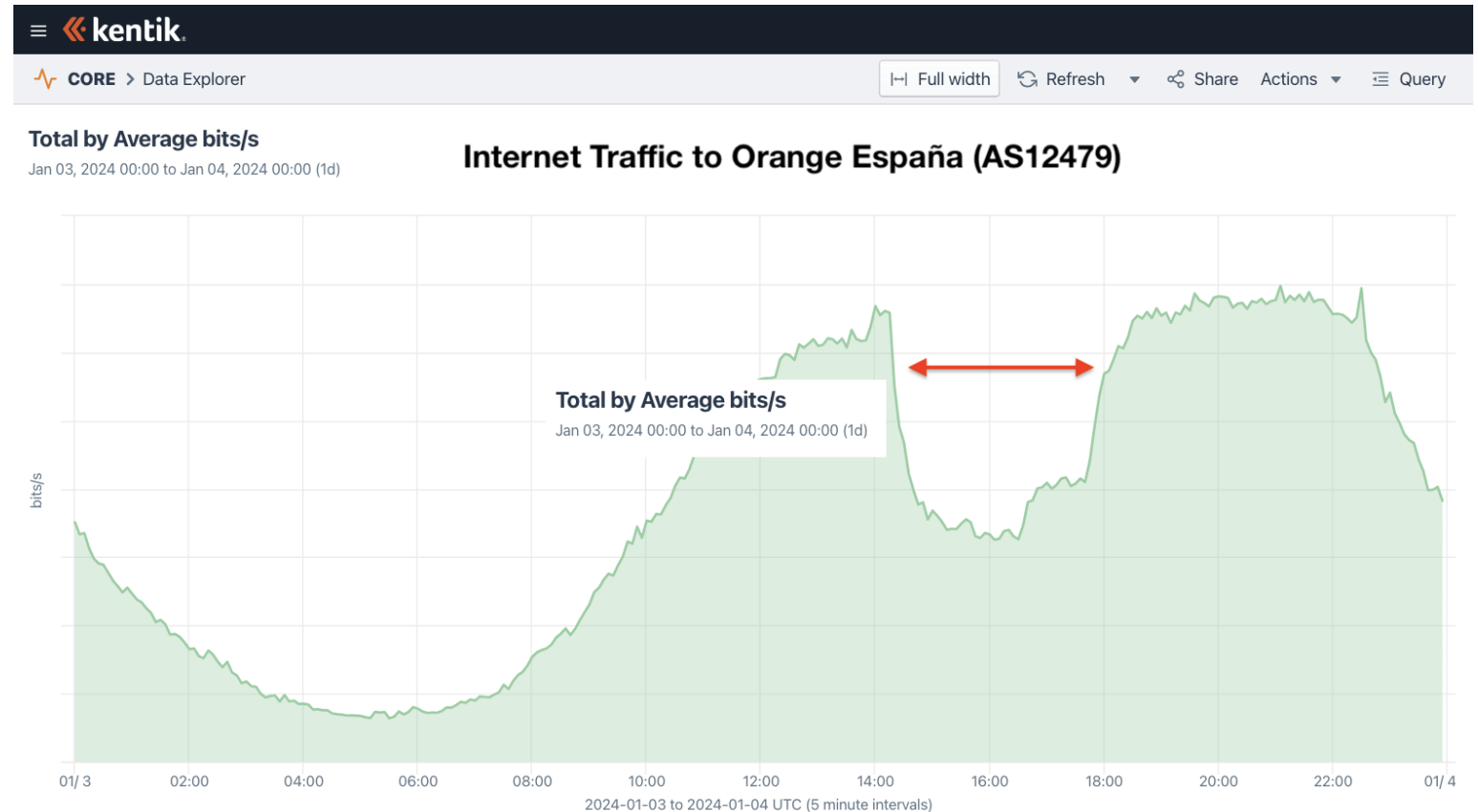
Cost and complexity reasons

Side remarks

Why did RIPE introduced 2-factor authentication?

The Orange Espana Outage

„Using a password found in a public leak of stolen credentials, a hacker was able to log into Orange España’s RIPE NCC portal using the password “ripeadmin.” *Oops!* Once in, this individual began altering Orange España’s RPKI configuration, rendering many of its BGP routes RPKI-invalid.“



<https://www.kentik.com/blog/digging-into-the-orange-espana-hack/>

The story begins at 09:28 UTC on January 3, when someone (presumably the attacker) began tinkering with publishing and revoking ROAs for IP ranges belonging to the Spanish mobile operator. Then, at 09:42 UTC they published three new ROAs for Orange España IP ranges with material impact.

Origin	prefix	maxLength	ta	expiration
AS12479	93.117.88.0/22	22	ripe	1704355258
AS12479	93.117.88.0/21	21	ripe	1704355258
AS12479	149.74.0.0/16	16	ripe	1704355258

[Source](#)

Given the fact that 93.117.88.0/22, 93.117.88.0/21, and 149.74.0.0/16 were all already originated by AS12479, those routes weren't affected, but 149.74.0.0/16 had quite a few more-specifics that were now going to be evaluated as RPKI-invalid due to the max prefix length setting of 16.

It wasn't until about 14:20 UTC (3:20pm local) that things got ugly. The attacker went for it and published four more ROAs with bogus origins. Two of the ROAs were /12's which covered over a thousand routes originated by AS12479 — all rendered RPKI-invalid by the publication of the following ROAs:

Origin	prefix	maxLength	ta	expiration
AS49581	85.48.0.0/12	12	ripe	1704355258
AS49581	90.160.0.0/12	12	ripe	1704355258
AS49581	93.117.88.0/21	21	ripe	1704355258
AS49581	145.1.232.0/21	21	ripe	1704355258

What happens this lecture?

IPv6

Motivation

Problem: IANA assigned the last IPv4 address block to RIRs in 2011



The screenshot shows the top of the NRO website. At the top, there are logos for NRO, AFRINIC, APNIC, ARIN, lacnic, and RIPE NCC. Below these is a navigation bar with links: ABOUT, ACCOUNTABILITY, INTERNET GOVERNANCE, TECHNICAL COORDINATION, POLICY, and DOCUMENT ARCHIVE. The main content area features the headline "Free Pool of IPv4 Address Space Depleted" with a date of "3 FEB 2011". The text below the headline states: "IPv6 adoption at critical phase" and "Montevideo, 3 February 2011 – The Number Resource Organization (NRO) announced today that the free pool of available IPv4 addresses is now fully depleted. On Monday, January 31, the Internet Assigned Numbers Authority (IANA) allocated two blocks of IPv4 address space to APNIC, the Regional Internet Registry (RIR) for the Asia Pacific region, which triggered a global policy to allocate the remaining IANA pool equally between the five RIRs. Today IANA allocated those blocks. This means that there are no longer any IPv4 addresses available for allocation from the IANA to the five RIRs." On the right side of the page, there is a "SEARCH" bar and a "QUICK LINKS" section with links to "NRO News", "ASO Review 2017", "Document Archives", "NRO FAQ", and "ASO Website".

Free Pool of IPv4 Address Space Depleted

3 FEB 2011

IPv6 adoption at critical phase

Montevideo, 3 February 2011 – The Number Resource Organization (NRO) announced today that the free pool of available IPv4 addresses is now fully depleted. On Monday, January 31, the Internet Assigned Numbers Authority (IANA) allocated two blocks of IPv4 address space to APNIC, the Regional Internet Registry (RIR) for the Asia Pacific region, which triggered a global policy to allocate the remaining IANA pool equally between the five RIRs. Today IANA allocated those blocks. This means that there are no longer any IPv4 addresses available for allocation from the IANA to the five RIRs.

SEARCH

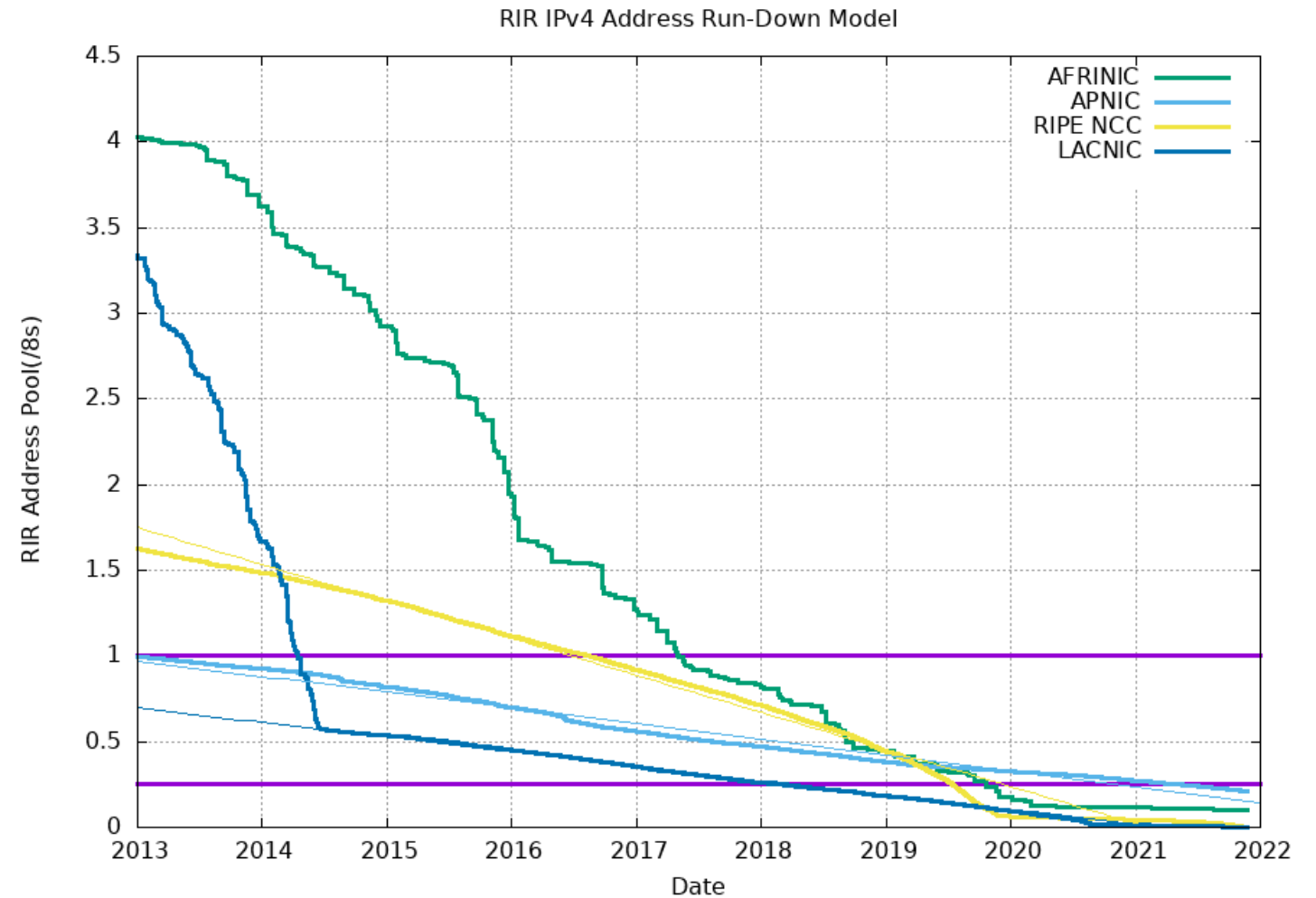
Search **SEARCH**

QUICK LINKS

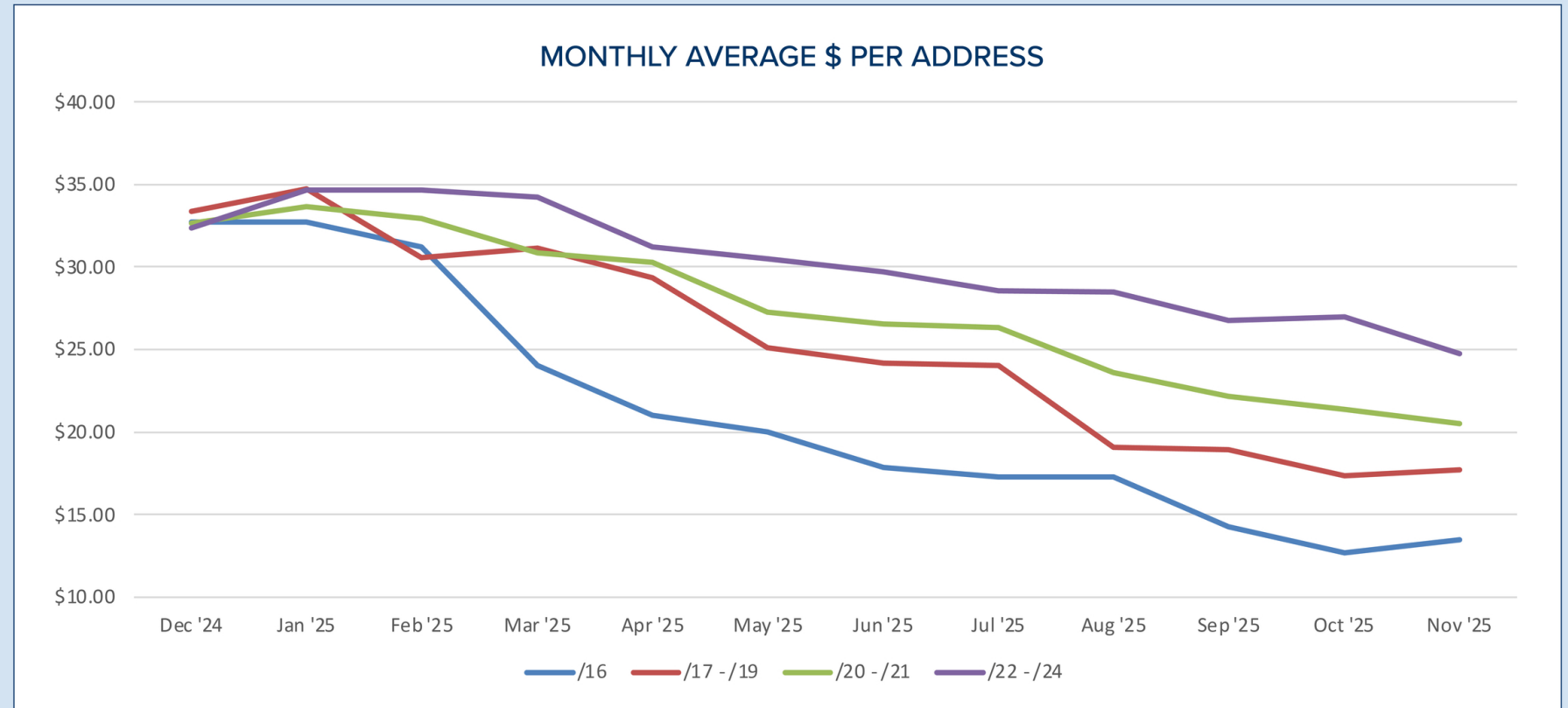
- NRO News
- ASO Review 2017
- Document Archives
- NRO FAQ
- ASO Website

“IPv4 Address Exhaustion” per RIR

Basically, **all address blocks have been assigned**. You may have a chance on the **waiting list**.

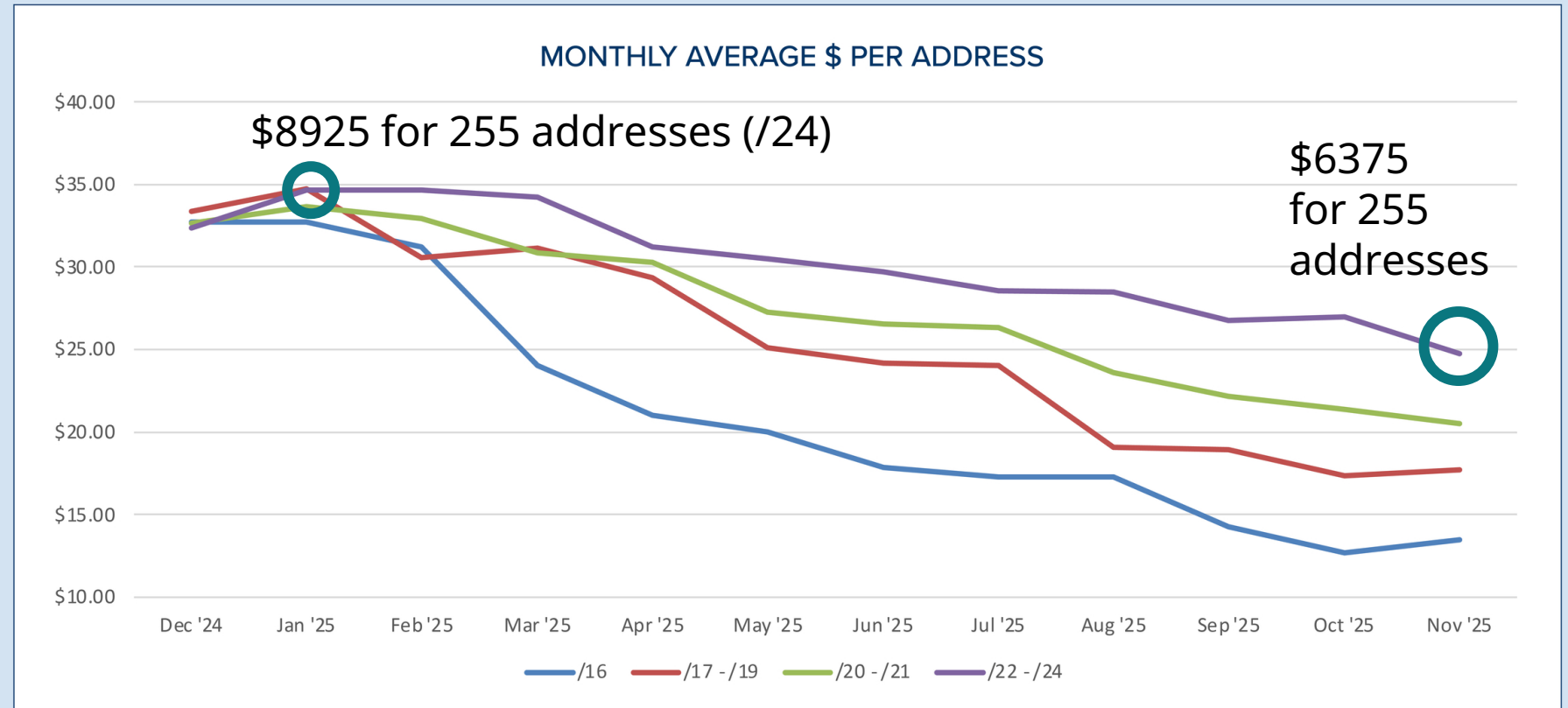


Source: <https://ipv4.potaroo.net/>



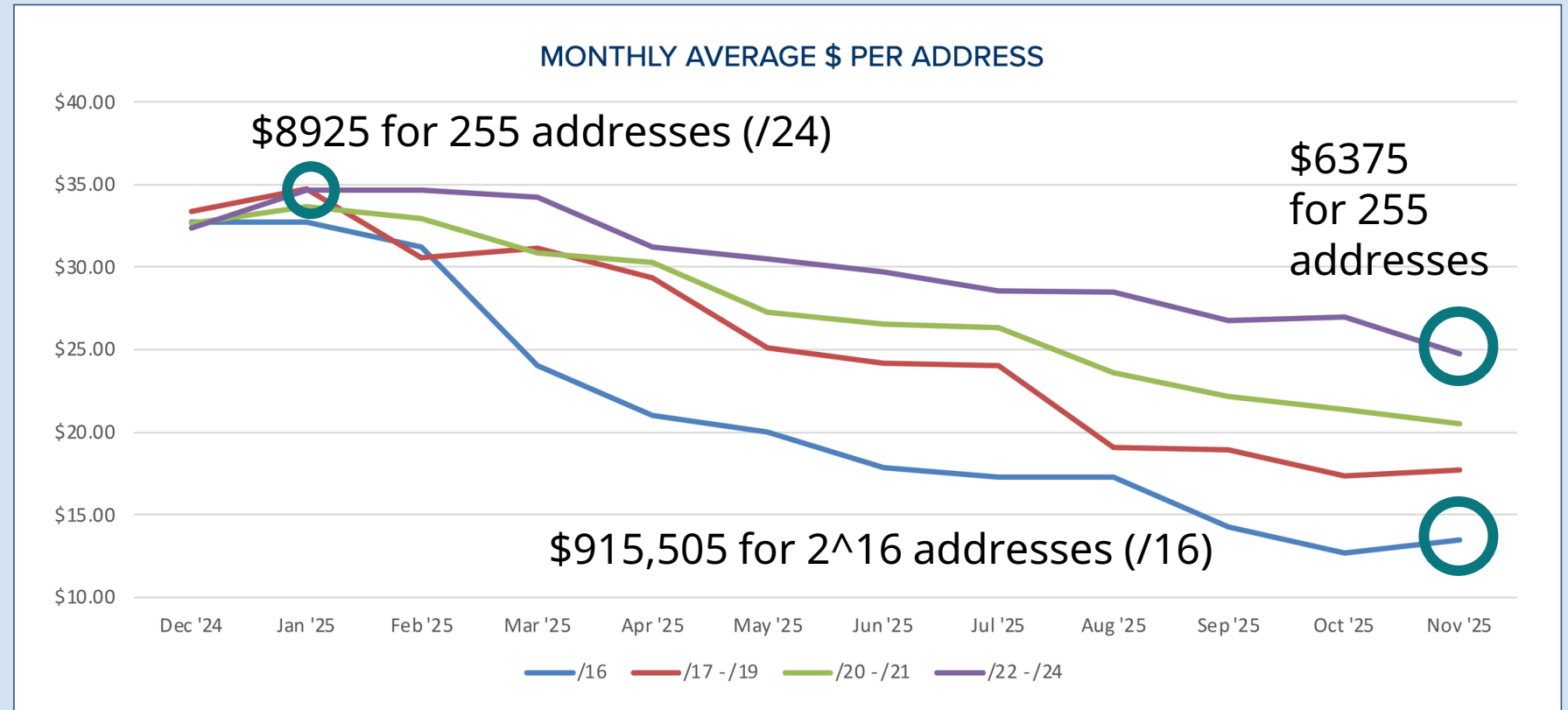
Report includes a mix of online and private sales. Does not include addresses from LACNIC. Some blocks may be part of a bundle with a negotiated price. Prices are reported as of date of agreement, not transfer.

Monthly
income



Report includes a mix of online and private sales. Does not include addresses from LACNIC. Some blocks may be part of a bundle with a negotiated price. Prices are reported as of date of agreement, not transfer.

Monthly
income



Report includes a mix of online and private sales. Does not include addresses from LACNIC. Some blocks may be part of a bundle with a negotiated price. Prices are reported as of date of agreement, not transfer.

End-To-End Arguments in System Design

J. H. SALTZER, D. P. REED, and D. D. CLARK

Massachusetts Institute of Technology Laboratory for Computer Science

This paper presents a design principle that helps guide placement of functions among the modules of a distributed computer system. The principle, called the end-to-end argument, suggests that functions placed at low levels of a system may be redundant or of little value when compared with the cost of providing them at that low level. Examples discussed in the paper include bit-error recovery, security using encryption, duplicate message suppression, recovery from system crashes, and delivery acknowledgment. Low-level mechanisms to support these functions are justified only as performance enhancements.

CR Categories and Subject Descriptors: C.0 [General] Computer System Organization—*system architectures*; C.2.2 [Computer-Communication Networks]: Network Protocols—*protocol architecture*; C.2.4 [Computer-Communication Networks]: Distributed Systems; D.4.7 [Operating Systems]: Organization and Design—*distributed systems*

General Terms: Design

Additional Key Words and Phrases: Data communication, protocol design, design principles

[ACM Transactions on Computer Systems, Nov. 1984, <https://dl.acm.org/doi/10.1145/357401.357402>]

End-To-End Arguments in System Design

J. H. SALTZER, D. P. REED, and D. D. CLARK

Massachusetts Institute of Technology Laboratory for Computer Science

This paper presents a design principle that helps guide placement of functions among the modules of a distributed computer system. The principle, called the end-to-end argument, suggests that functions

NAT is not a solution for a proper Internet.

architectures; C.2.2 [**Computer-Communication Networks**]: Network Protocols—*protocol architecture*; C.2.4 [**Computer-Communication Networks**]: Distributed Systems; D.4.7 [**Operating Systems**]: Organization and Design—*distributed systems*

General Terms: Design

Additional Key Words and Phrases: Data communication, protocol design, design principles

[ACM Transactions on Computer Systems, Nov. 1984, <https://dl.acm.org/doi/10.1145/357401.357402>]

IPv6 address space

2^{128} Bit

Notation of IPv6 addresses

2^{128} Bit

X 16 Bit Hex

X : X : X : X : X : X : X : X : X

Example

2001 : 0DB8 : 0000 : 0000 : 0000 : 0000 : 0000 : 0001

Compact syntax

Subsequent blocks
of zeros can be
replaced by ::

2001:0DB8::0001

Leading zeros can
be removed

2001:0DB8:0000:0000:0000:0000:0000:**1**

Example

2001:0DB8:0000:0000:0000:0000:0000:0001

2001:DB8::1

IPv6 Innovations: Simpler **addressing** and **routing**

Elimination of address bottleneck: 128 Bit addresses

Several addresses per interface are common

IPv6 Innovations: Simpler protocol architecture

Slim, fixed header for fast processing

Optional extension headers

Format framework for header classes

No header checksum (already done by layer 2 and 4)

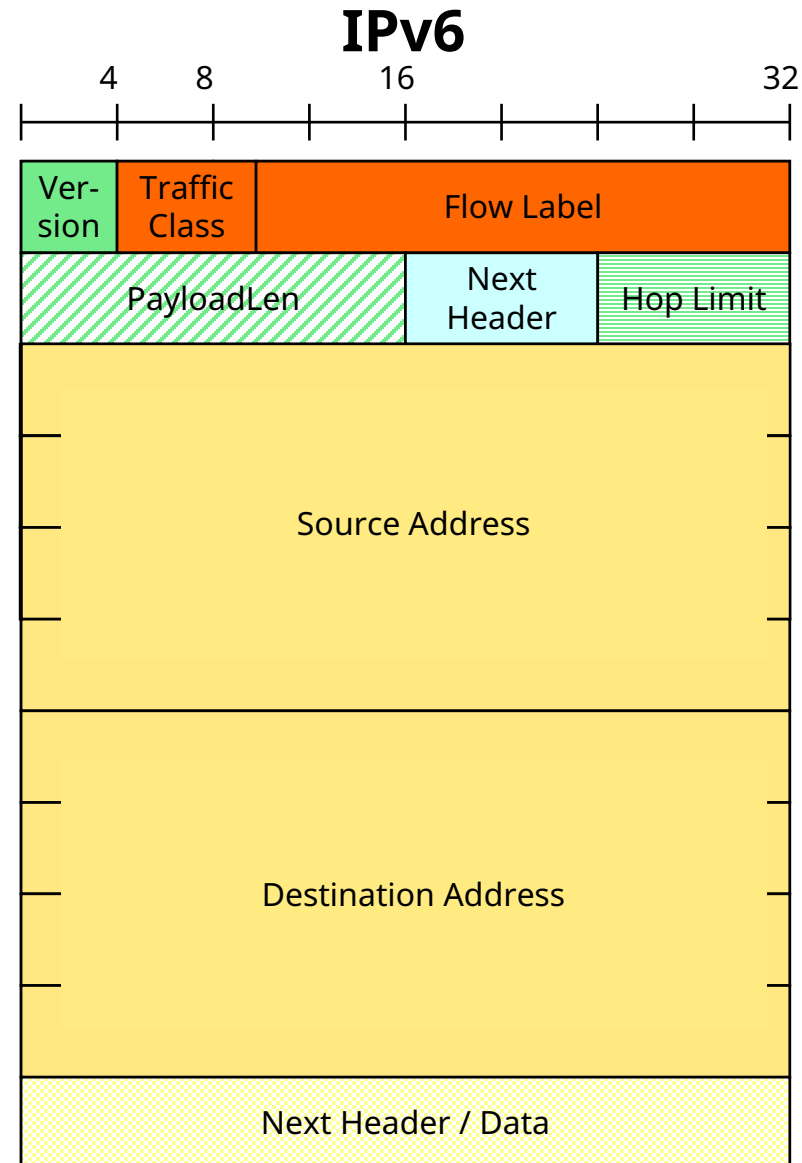
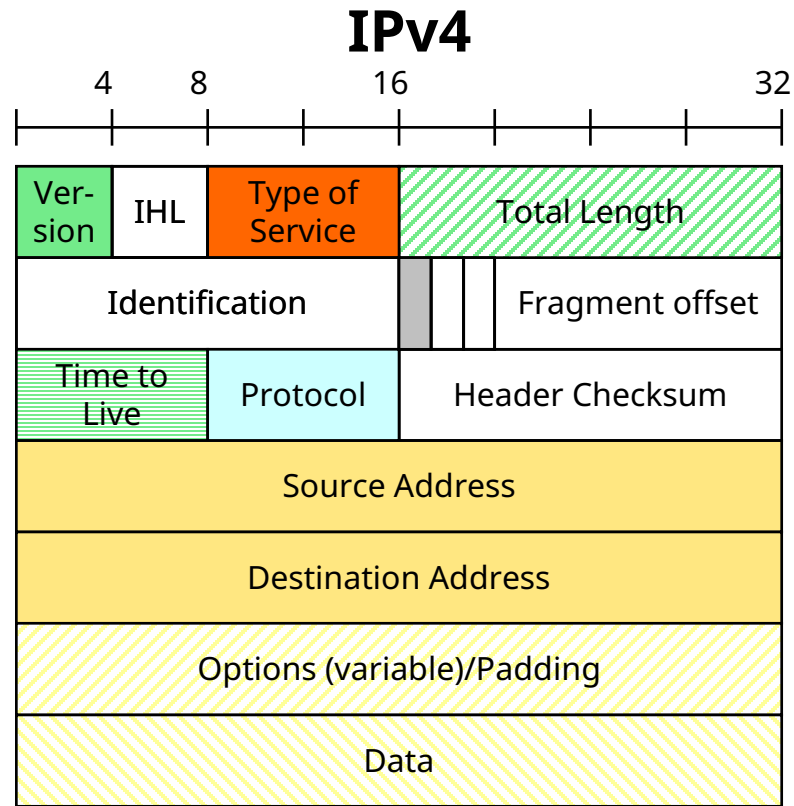
No fragmentation in routers

IPv6 Innovations: Simpler **administration**

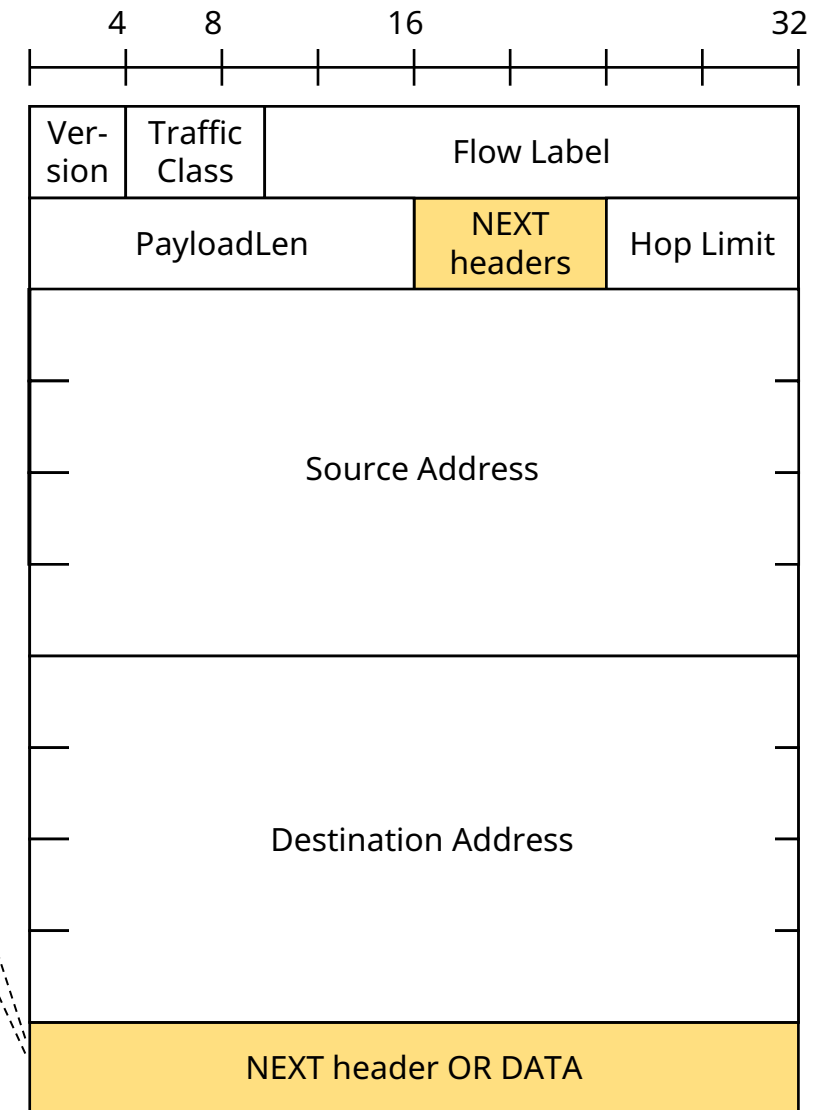
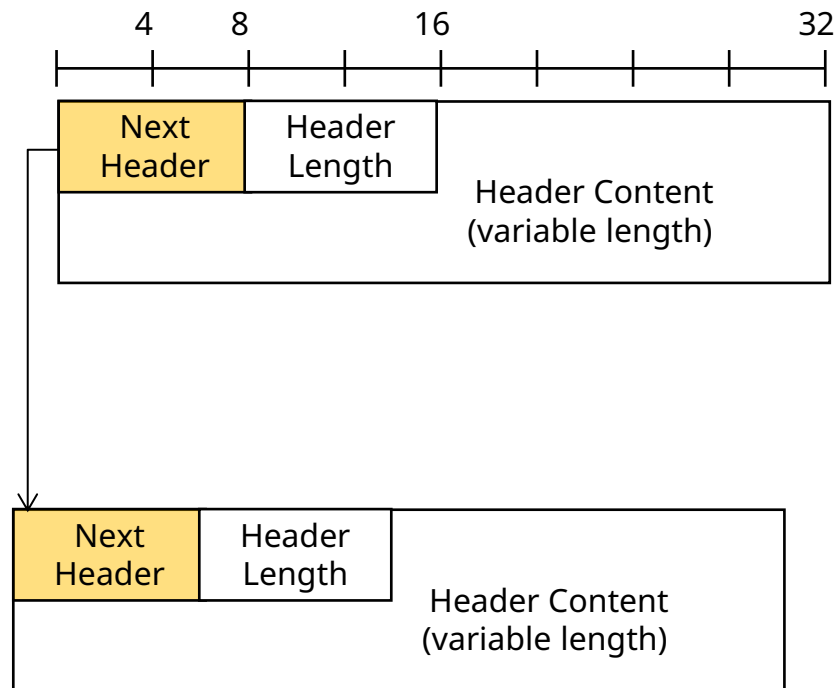
Stateless Address Autoconfiguration (SLAAC) of interfaces without DHCPv6

Renumbering via prefix change

Packet Formats: IPv6 vs. IPv4



IPv6 extension headers



Six basic extension headers

1. Routing

Definition of a full or partly specified route

Attention: This header is deprecated since December 2007 (see RFC 5095)

2. Fragmentation

Fragmentation / defragmentation of frames

Difference with IPv4: Only the source can do fragmentation.

Routers for which a packet is too large, only send an error message back to the source.

3. Authentication

Security information, authentication of the sender

4. Encapsulation

Tunneling, e.g., for encrypted data

5. Hop-by-Hop options

Dedicated options to be processed by every router and host, information for single links

Momentarily only the support of Jumbograms (packets exceeding the normal IP packet length) is defined (length specification).

6. Destination options

Additional information for the destination

Compared with IPv4, IPv6 header is simpler

removed

- Fragmentation
- the checksum
- header length

reason

leave problems
to the end host

simplify handling

Added ...

- new options mechanism
- expanded addresses
- flow label

simplify handling

flexibility

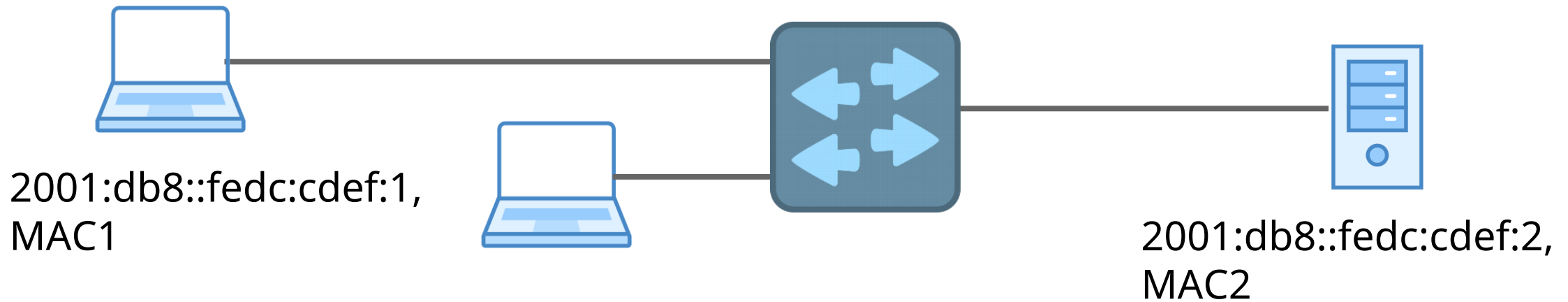
In IPv6, ICMP is much more crucial for operation

How does neighbor discovery work?

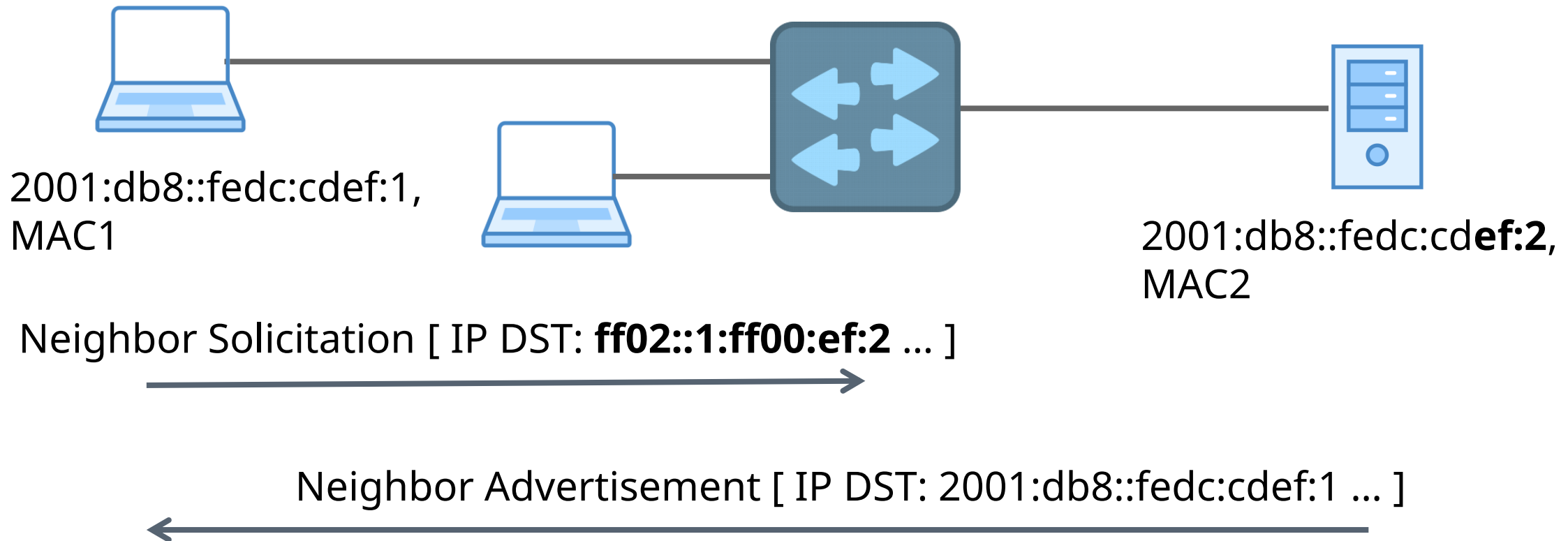
How does router discovery work?

Filtering ICMP harms.

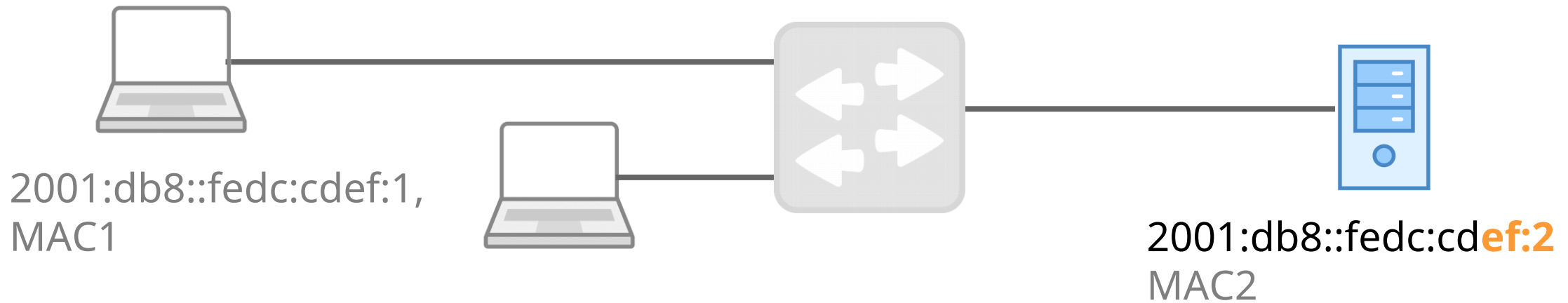
IPv6 Neighbor Discovery Protocol



IPv6 Neighbor Discovery Protocol. ND uses Multicast



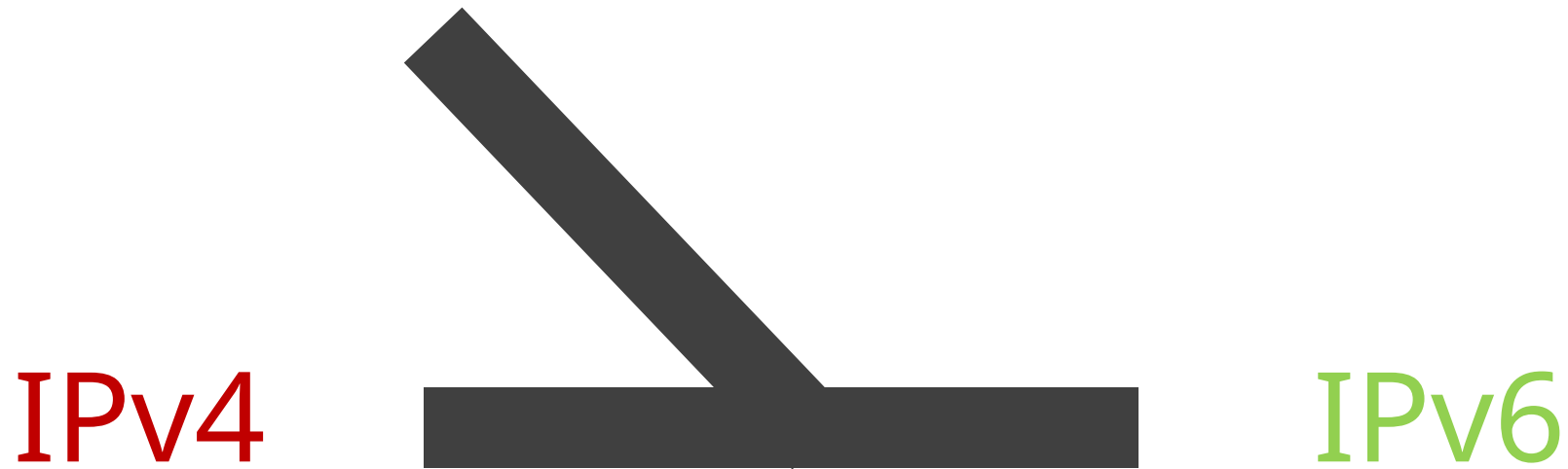
IPv6 Neighbor Discovery Protocol. ND uses Multicast



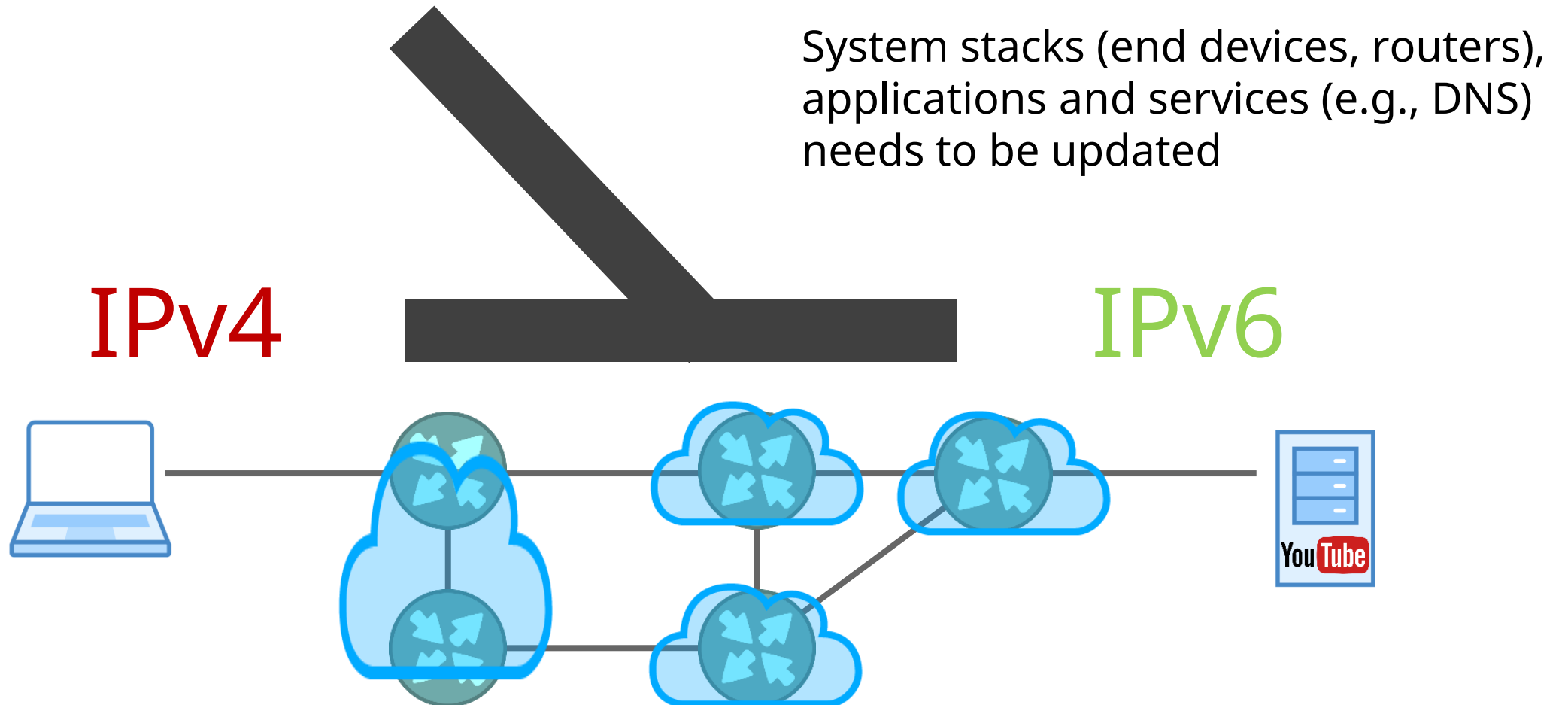
Neighbor Solicitation [IP DST: **ff02::1:ff00:ef:2**]

Solicited Node Multicast Address: ff02::1:ff**00:0000**/104
Remaining 24 bits encode the last 24 bits of the requested IPv6 address.

We cannot replace IPv4 by night



We cannot replace IPv4 by night



Transition from IPv4 to IPv6

Porting

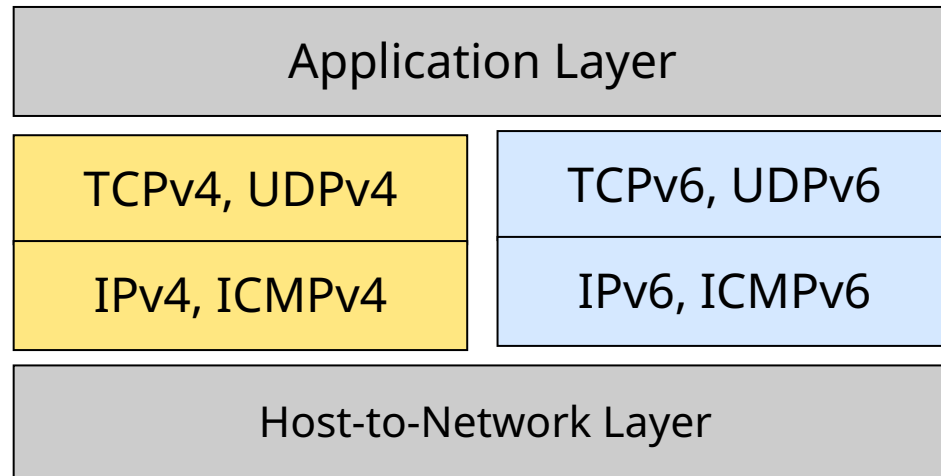
Make applications IPv6-ready by using the new sockets API, new address structures

Migration

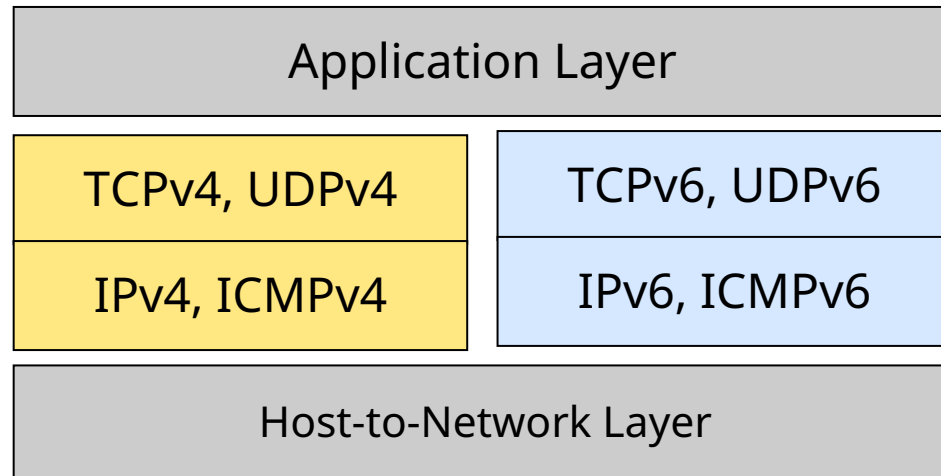
Incremental change from v4 to v6; temporarily use both protocols in parallel by deploying dual stack, tunnels, protocol translators

Migration by dual stacks

Run both stacks in parallel and use what is available!



Migration by dual stacks. Run both stacks in parallel and use what is available!



Upon activation of IPv6, IPv4 can continue being used (multi protocol approach)

Devices can keep their addresses (IPv4 in IPv6)

Application / libraries choose the IP version

Depending on DNS answer with IPv6 preference and received IP packets

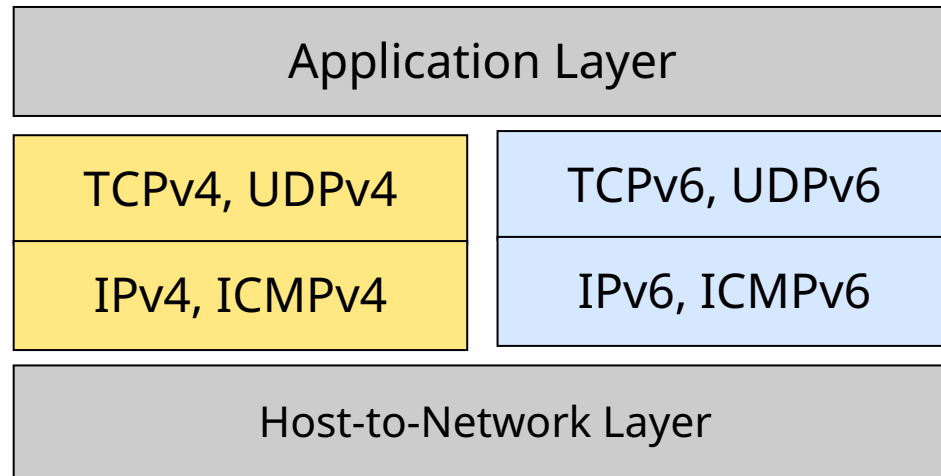
Dual stack operation can continue indefinitely, allowing step by step porting of applications

But requires an increasing use of IPv4 NATTING (recursive)

Problems result from inconsistencies of Network / DNS configurations

Example: DNS returns IPv4 and IPv6 address, application selects v4 but v4 is already disabled

Migration by dual stacks. Run both stacks in parallel and use what is available!



Check „Happy Eyeballs Version 2: Better Connectivity Using Concurrency “ RFC 8305. (draft-ietf-happy-happyeyeballs-v3 is coming)

Upon activation of IPv6, IPv4 can continue being used (multi protocol approach)
Devices can keep their addresses (IPv4 in IPv6)

Application / libraries choose the IP version
Depending on DNS answer with IPv6 preference and received IP packets

Dual stack operation can continue indefinitely, allowing step by step porting of applications
But requires an increasing use of IPv4 NATTING (recursive)

Problems result from inconsistencies of Network / DNS configurations
Example: DNS returns IPv4 and IPv6 address, application selects v4 but v4 is already disabled

Happy Eyeballs, RFC 8305

„This document defines the algorithm for "Happy Eyeballs", a technique“.

The approach consists of several distinct phases:

1. Initiation of asynchronous DNS queries
2. Sorting of resolved destination addresses
3. Initiation of asynchronous connection attempts
4. Establishment of one connection, which cancels all other attempts

Happy Eyeballs, RFC 8305

Resolution Delay: The time to wait for a AAAA response after receiving an A response. **Recommended to be 50 milliseconds.**

First Address Family Count: The number of addresses belonging to the first address family (such as IPv6) that should be attempted before attempting another address family. **Recommended to be 1; 2 may be used to more aggressively favor a particular address family.**

Connection Attempt Delay: The time to wait between connection attempts in the absence of RTT data. **Recommended to be 250 milliseconds.**

Minimum Connection Attempt Delay: The minimum time to wait between connection attempts. **Recommended to be 100 milliseconds. MUST NOT be less than 10 milliseconds.**

Maximum Connection Attempt Delay: The maximum time to wait between connection attempts. **Recommended to be 2 seconds.**

Migration by dual stacks. Run both stacks in parallel and use what is available!

Upon activation of IPv6, IPv4 can continue being used (multi protocol approach)

Devices can keep their addresses (IPv4 in IPv6)

Application / libraries choose the IP version
Depending on DNS answer with IPv6 preference
and selected IP address

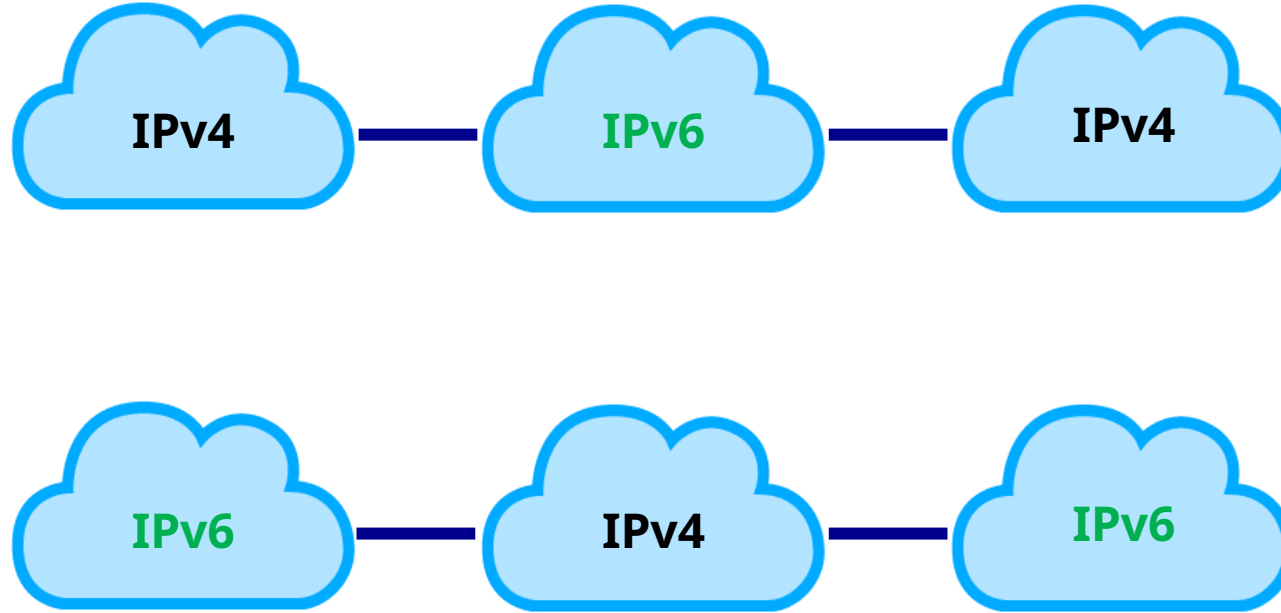
Dual stack operation can continue indefinitely, allowing step by step porting of applications
But requires an increasing use of IPv4 NATTING (recursive)

Problems result from inconsistencies of Network / DNS configurations
Example: DNS returns IPv4 and IPv6 address, application selects v4 but v4 is already disabled

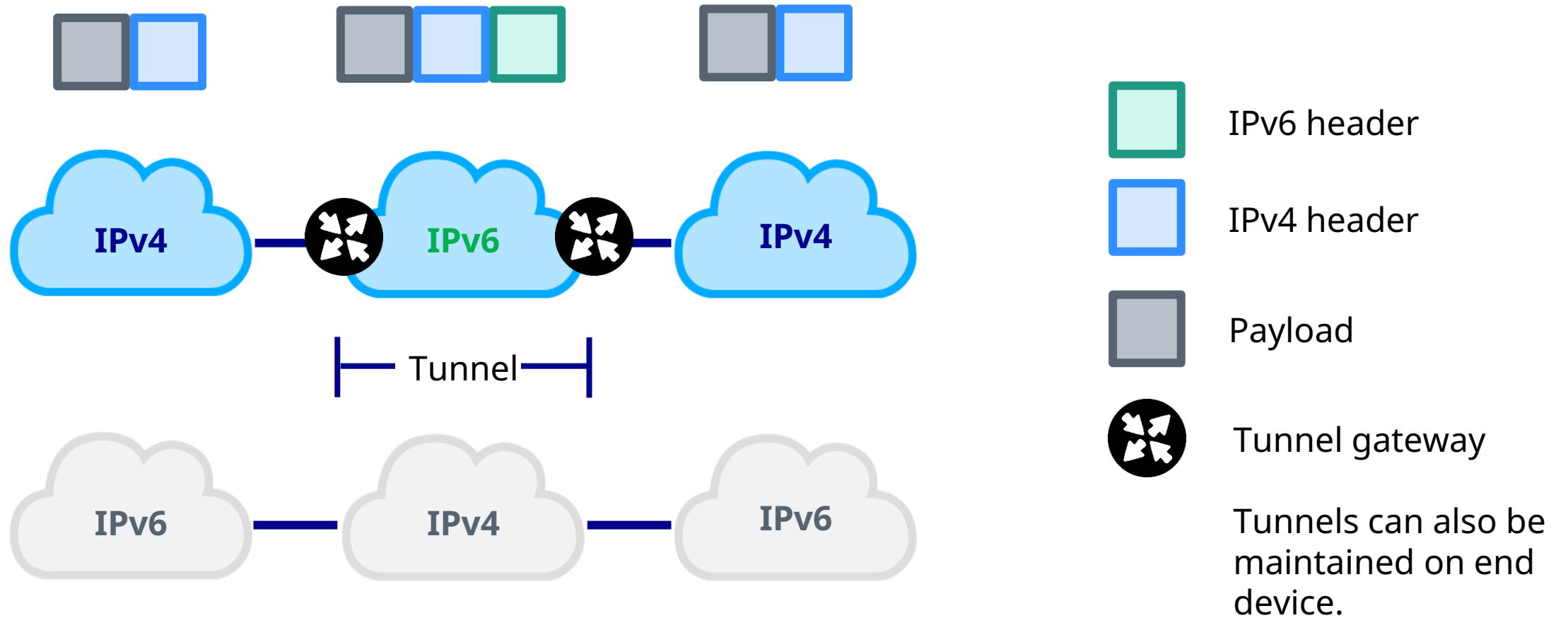
Does this work in all scenarios?

Check „Happy Eyeballs Version 2: Better Connectivity Using Concurrency “ RFC 8305. (draft-ietf-happy-happyeyeballs-v3 is coming)

Dual stack does not help in all scenarios



Dual stack does not help in all scenarios. Migration by tunneling.



Challenges when tunneling

Learning the tunnel end point

Which v6 (or v4) tunnel endpoint is responsible for which v4 (or v6) IP address?

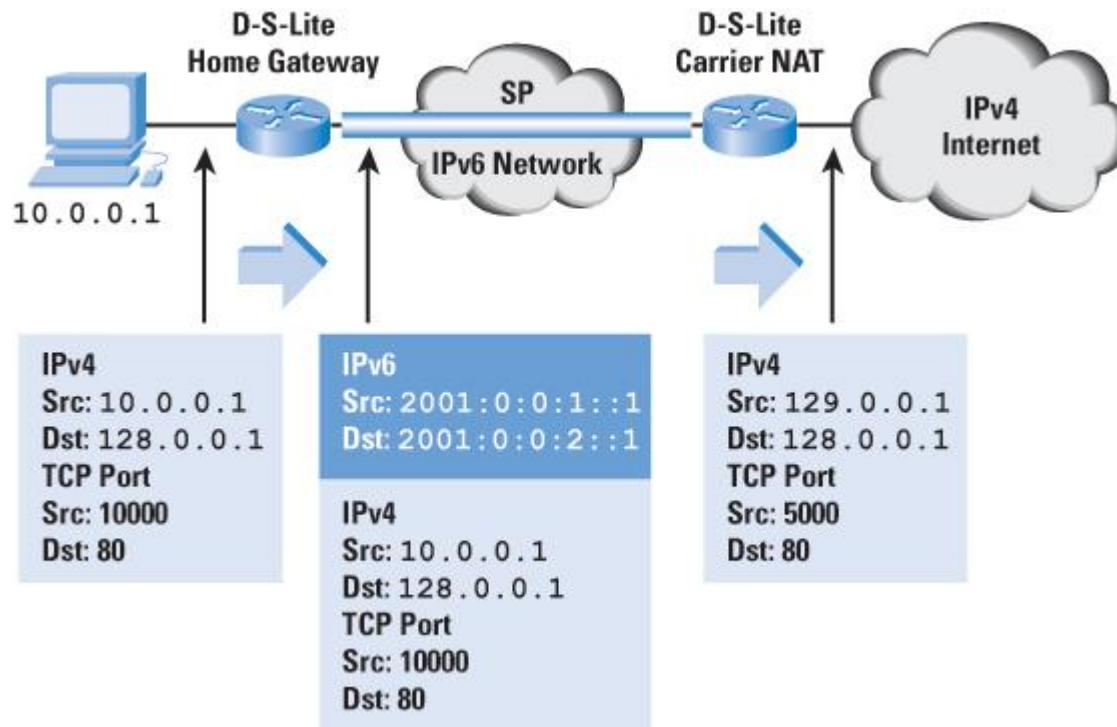
More overhead because of two headers

Additional headers increase packets sizes

Increased packet sizes may lead to fragmentation

Packet fragmentation introduces additional challenges

Tunnel approach: Dual Stack Lite (RFC 6333)



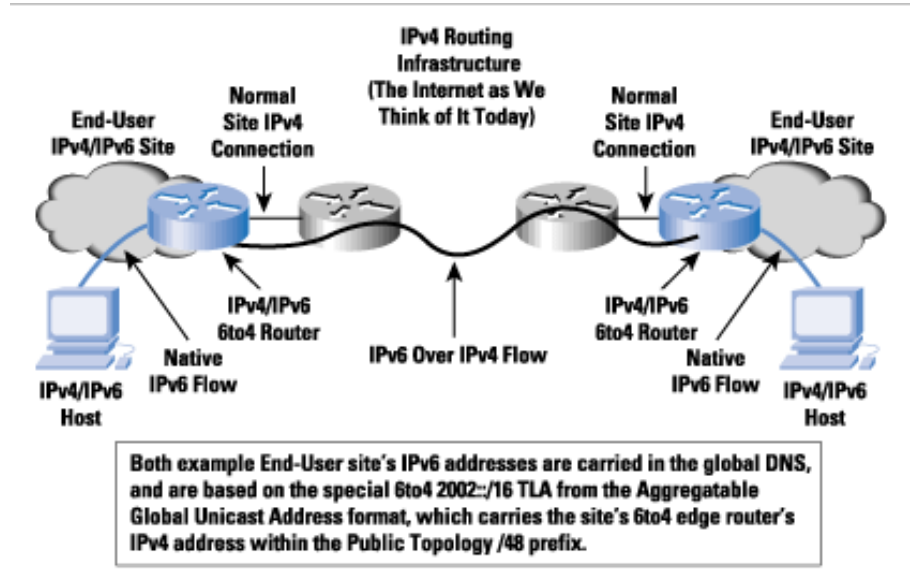
Source: Geoff Huston, "NAT++: Address Sharing in IPv4", The Internet Protocol Journal, Volume 13, No.2

Starting point: No public IPv4 at the customer because of v4 exhaustion

Idea: Tunneling + NAT

- IPv4 over IPv6 tunnel to Carrier-grade IPv4-IPv4 NAT
- Infrastructure of the provider can be migrated to IPv6
- Enables many customers to share a single globally unique IPv4 address, saves public IPv4 addresses

Tunnel approach: 6-to-4 tunneling



Source: Brian E. Carpenter et al., "Routing IPv6 over IPv4", The Internet Protocol Journal, Volume 13, No.1

Starting point: Allow IPv6 islands to automatically interconnect, with minimized manual configuration

Idea: Automated point-to-multipoint tunnels (RFC 3056)

- Assign an IPv6 prefix to each IPv4 address, DNS will return both, sending host will use IPv6
- Nearest router identifies 6to4 packet based on special IPv6 prefix and encapsulates in IPv4

6to4 IPv6 /48 prefix

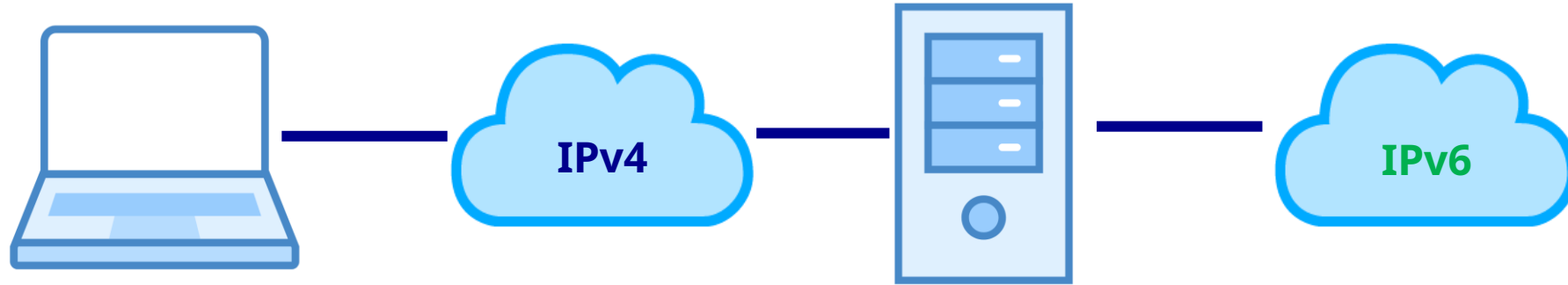
20 02 | a0 00 | 02 01

16

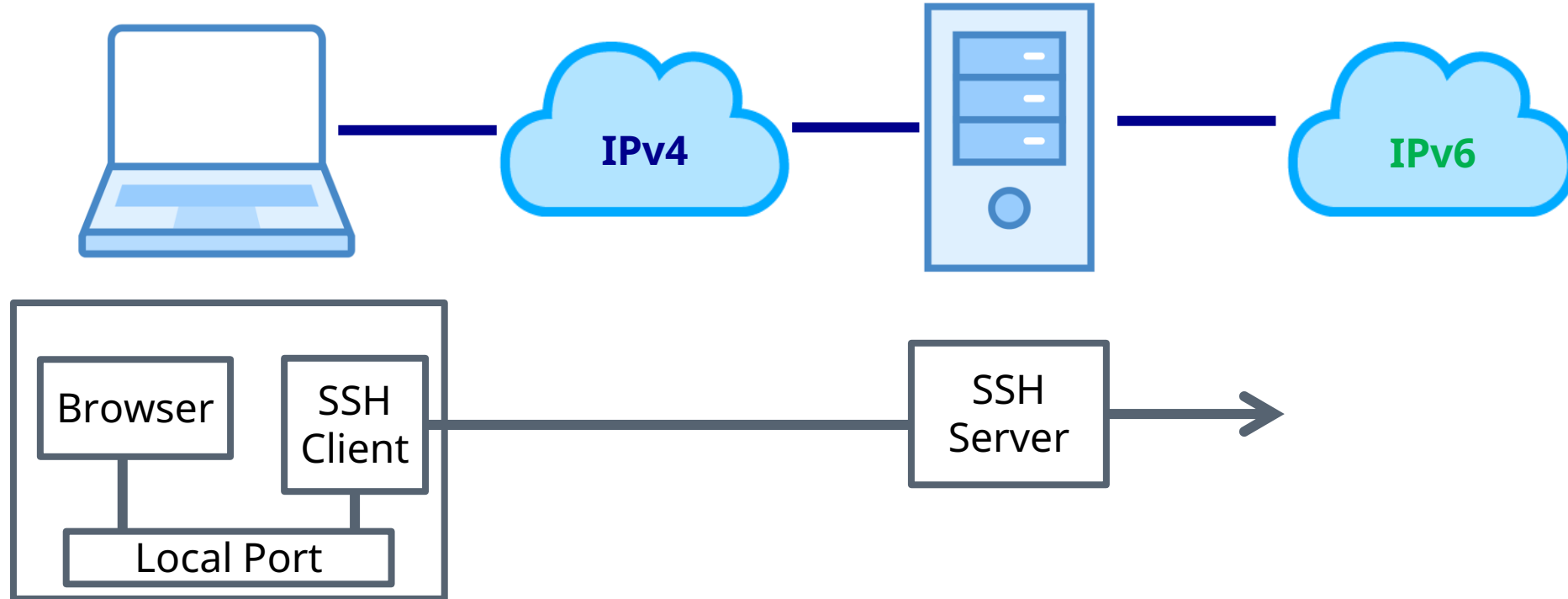
32

48

SSH IPv6 Tunnel (port forwarding) + SOCKS Proxy



SSH IPv6 Tunnel (port forwarding) + SOCKS Proxy

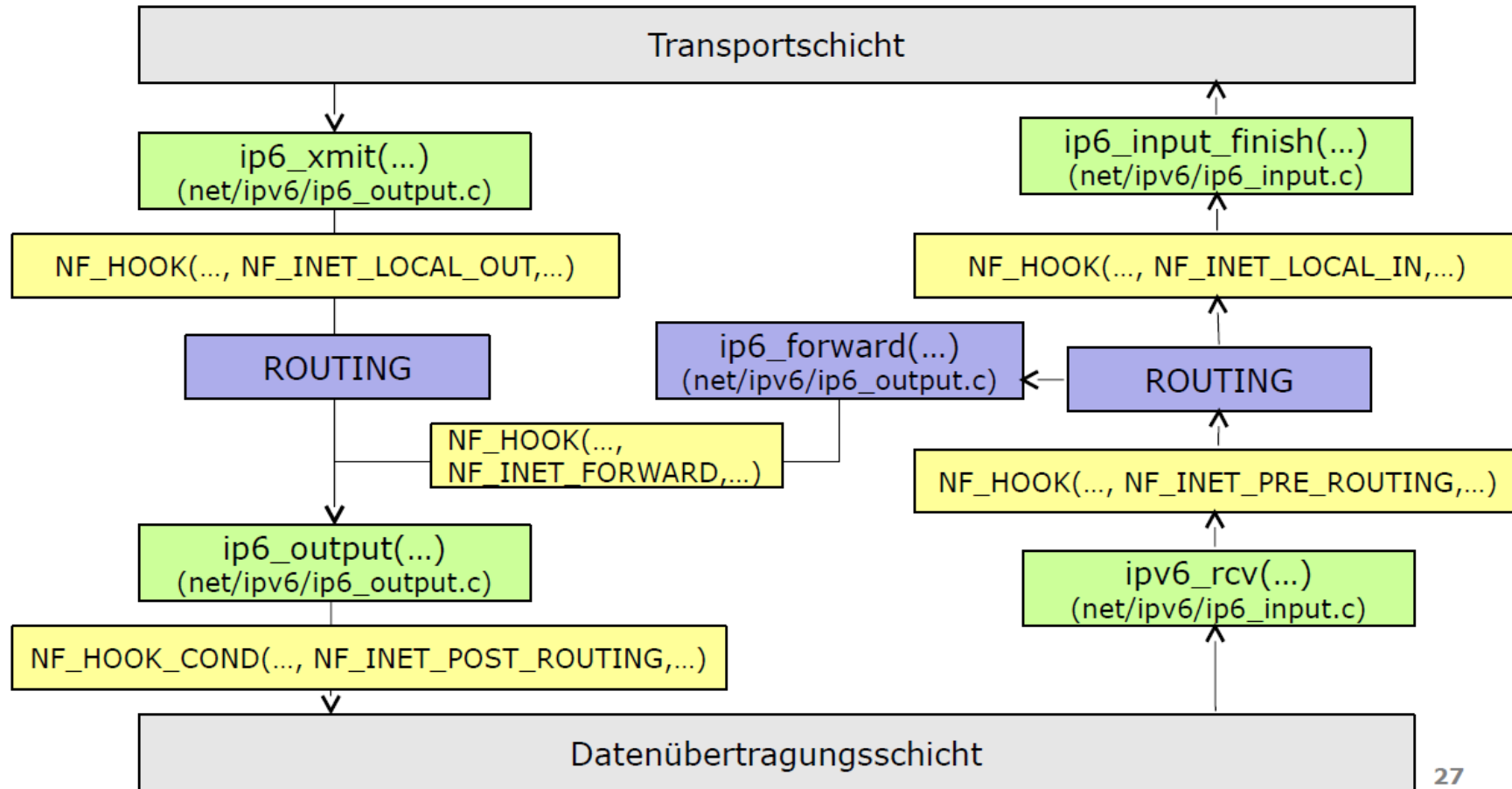


SSH client opens TCP port locally and connects to the SSH server
Browser connects to local TCP using SOCKS Proxy protocol

SSH IPv6 Tunnel (port forwarding) + SOCKS Proxy

1. You open an SSH connection to a remote server. As you open that connection, your SSH client will also open a local TCP port, available only to your computer. In this example, in our example local TCP port :9090.
2. You configure your browser (Chrome/Firefox/...) to use that local proxy instead of directly going out on the internet.
3. The remote SSH server accepts your SSH connection and will act as the outgoing proxy_/vpn_ for that SOCKS5 connection.
4. To start such a connection, run the following command in your terminal.
`ssh -D 9090 -q -N user@host`
5. **Enable SOCKS proxy in your browser to use localhost and port 9090, e.g.,**
`chrome --proxy-server="socks5://localhost:9090"`

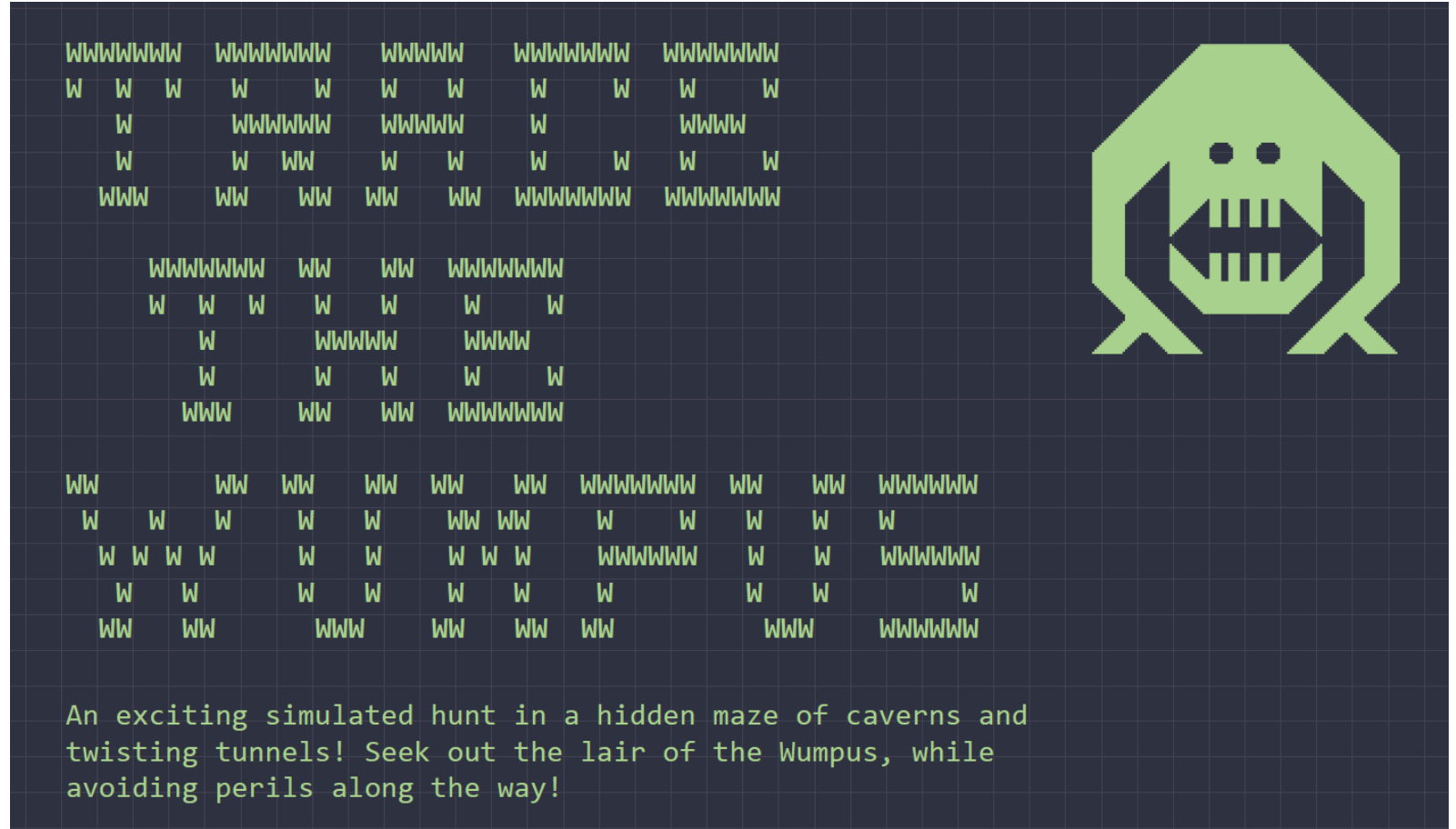
IPv6 implementation in the Linux kernel



27

You will experience IPv6 in practice in the lab by hunting the Wumpus.

Trace the Wumpus. In collaboration with Leitwert.



The new Wumpus game. Frontend.

[GOAL] Use your local traceroute tool and play

```
~$ traceroute6 play.wumpus.quest
```

```
traceroute to play.wumpus.quest (2a06:2904::10:10:11), 30 hops max, 80 byte packets
```

1	W-----W	(2a06:2907::ff:ff:cb)	6.285 ms
2	W--TRACE.THE.WUMPUS-----W	(2a06:2907::ff:ff:c4)	12.055 ms
3	W-----W	(2a06:2907::ff:ff:cb)	17.244 ms
4	W--YOU.ARE.IN.ROOM-14-----W	(2a06:2907::ff:ff:95)	22.157 ms
5	W--TUNNELS.LEAD.TO-4-13-15-----W	(2a06:2907::ff:ff:b3)	28.317 ms
6	W-----W	(2a06:2907::ff:ff:cb)	32.640 ms
7	W--MOVE---move.ROOM.wumpus.quest-----W	(2a06:2907::ff:ff:c9)	29.382 ms
8	W--SHOOT--shoot.ROOM1.ROOMX.wumpus.quest-----W	(2a06:2907::ff:ff:ca)	29.474 ms
9	W-----W	(2a06:2904::10:10:11)	29.743 ms

```
~$ traceroute6 move.15.wumpus.quest
```

```
...
```

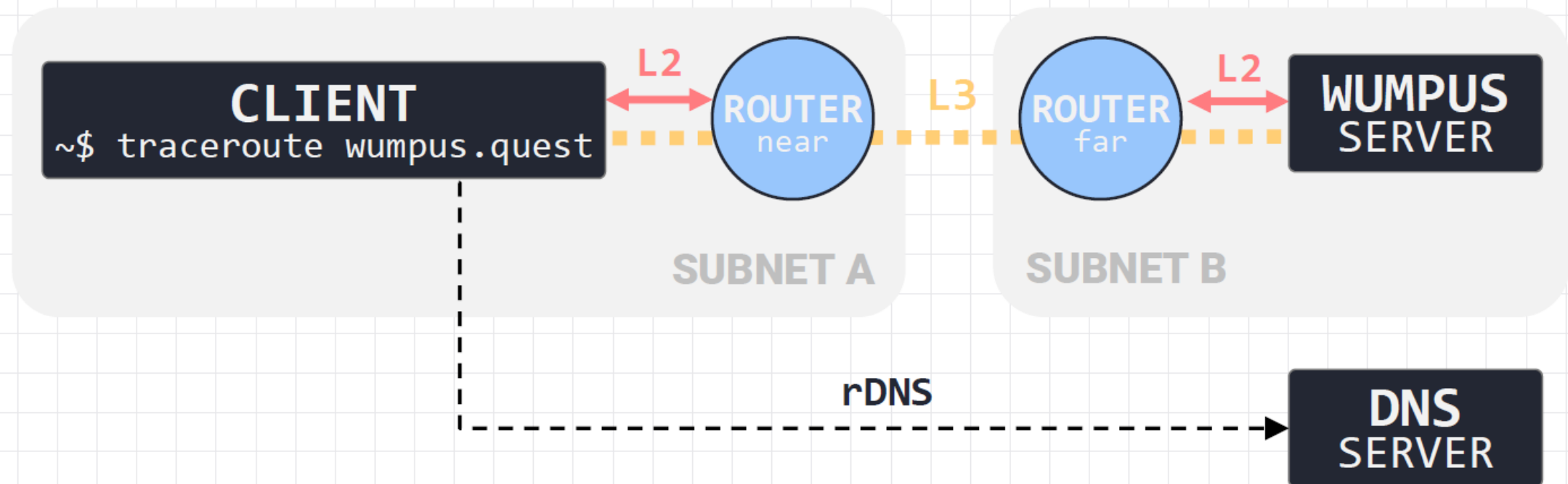
```
~$ traceroute6 shoot.16.20.13.wumpus.quest
```

```
...
```

The new Wumpus game. Architecture.

We are building a server component

- Respond to packets with TTL exceeded from multiple IP addresses
- We need nothing else (no routing, no client, no DNS) – only packet crafting on layers 2-7



The new Wumpus game. Architecture.

We are building a server component

- Respond to packets with TTL exceeded from multiple IP addresses
- We need nothing else (no routing, no client, no DNS) – only packet crafting on layers 2-7

